

■ Panduan Belajar

Laravel 12

Step by Step untuk Pemula

■ Tahun	2025–2026
■ Estimasi	8–10 Minggu
■ Total Fase	5 Fase Pembelajaran
■ Tools	PHP 8.2+, Composer, Laravel 12, MySQL/SQLite

Panduan ini membawa kamu dari instalasi tools hingga deploy aplikasi Laravel pertamamu.

Daftar Isi

Fase 1 — Persiapan & Instalasi

- Install PHP 8.2+, Composer, Node.js
- Setup project Laravel 12
- Konfigurasi .env & database

Fase 2 — Konsep Dasar Laravel

- Routing — peta URL aplikasi
- Controller — logika terpisah
- Blade Templates — tampilan HTML elegan

Fase 3 — Database & Eloquent ORM

- Migration — struktur tabel
- Model Eloquent — CRUD mudah
- Relasi database: hasMany, belongsTo

Fase 4 — Fitur Penting

- Authentication dengan Laravel Breeze
- Middleware & keamanan
- File upload & Storage

Fase 5 — Build Proyek Pertama

- CRUD Application lengkap
- Search, Pagination, Filter
- Deploy ke hosting

Langkah 1.1 — Install PHP 8.2+

Laravel 12 membutuhkan PHP versi 8.2 ke atas. Pilih cara instalasi sesuai sistem operasimu:

Windows — Laragon (direkomendasikan):

- Download Laragon Full di laragon.org/download
- Install seperti biasa — PHP sudah langsung tersedia
- Tidak perlu konfigurasi PATH secara manual

Mac — Homebrew:

```
brew install php
```

Cek versi PHP:

```
php -v  
# Output: PHP 8.2.x (cli) atau lebih tinggi
```

Langkah 1.2 — Install Composer

Composer adalah package manager PHP. Laravel diinstall melalui Composer.

Windows:

Download Composer-Setup.exe dari getcomposer.org, ikuti wizard instalasi.

Mac/Linux:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer
```

Cek Composer:

```
composer --version  
# Output: Composer version 2.x.x
```

Langkah 1.3 — Install Node.js

Node.js dibutuhkan untuk mengelola aset frontend (CSS/JS) via Vite yang built-in di Laravel 12.

- Download versi LTS di nodejs.org
- Install seperti biasa (Windows/Mac/Linux)
- Cek: `node -v` (v20+) dan `npm -v` (v10+)

Langkah 1.4 — Buat Project Laravel 12

```
composer create-project laravel/laravel nama-projectku
cd nama-projectku
php artisan serve  
# Buka: http://localhost:8000
```

■ Jika ketiga tools (PHP, Composer, Node.js) sudah terinstall, jalankan `php -v && composer --version && node -v` untuk verifikasi sekaligus.

Langkah 1.5 — Konfigurasi .env & Database

File .env adalah konfigurasi rahasia project. Jangan upload ke GitHub!

SQLite (termudah untuk pemula):

```
# Di file .env:
DB_CONNECTION=sqlite

# Buat file database:
touch database/database.sqlite
```

MySQL (via Laragon/XAMPP):

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=belajar_laravel
DB_USERNAME=root
DB_PASSWORD=
```

Jalankan Migration:

```
php artisan migrate
# Output: DONE pada semua migration = sukses!
```

■ Untuk belajar, mulai dengan SQLite dulu — tidak perlu install MySQL terpisah.

Langkah 2.1 — Routing

Routing adalah peta URL aplikasi. Semua route ada di routes/web.php.

```
// Route paling sederhana
Route::get('/halo', function () {
    return 'Halo Dunia!';
});

// Route dengan parameter
Route::get('/produk/{id}', function ($id) {
    return 'Produk ID: ' . $id;
});

// Named route
Route::get('/profil', [ProfilController::class, 'index'])
    ->name('profil');

// Satu baris untuk semua CRUD
Route::resource('produk', ProdukController::class);

// Cek semua route
// php artisan route:list
```

Tabel HTTP Methods:

Method	URL Contoh	Fungsi
GET	/produk	Menampilkan data
POST	/produk	Menyimpan data baru
PUT	/produk/{id}	Mengubah data
DELETE	/produk/{id}	Menghapus data

Langkah 2.2 — Controller

Controller memisahkan logika dari routing agar kode lebih terstruktur.

```
# Buat resource controller
php artisan make:controller ProdukController --resource

# Buat model + migration + controller sekaligus
php artisan make:model Produk -mcr

// app/Http/Controllers/ProdukController.php
class ProdukController extends Controller
{
    public function index()
    {
        $produk = Produk::all();
        return view('produk.index', compact('produk'));
    }
}
```

```

    }

    public function store(Request $request)
    {
        $request->validate([
            'nama' => 'required|string|max:255',
            'harga' => 'required|numeric|min:0',
        ]);
        Produk::create($request->validated());
        return redirect()->route('produk.index')
            ->with('sukses', 'Produk berhasil ditambahkan!');
    }

    public function destroy(Produk $produk)
    {
        $produk->delete();
        return redirect()->route('produk.index');
    }
}

```

7 Method Resource Controller:

Method	URL	HTTP	Fungsi
index()	/produk	GET	Tampilkan semua
create()	/produk/create	GET	Form tambah baru
store()	/produk	POST	Simpan data baru
show(\$id)	/produk/{id}	GET	Tampilkan satu
edit(\$id)	/produk/{id}/edit	GET	Form edit
update(\$id)	/produk/{id}	PUT	Simpan perubahan
destroy(\$id)	/produk/{id}	DELETE	Hapus data

Langkah 2.3 — Blade Templates

Blade adalah template engine Laravel untuk membuat halaman HTML dengan logika PHP yang elegan.

```

{{-- Layout induk: resources/views/layouts/app.blade.php --}}
<!DOCTYPE html>
<html lang="id">
<head><title>@yield('judul', 'Aplikasi')</title></head>
<body>
    <nav>...</nav>
    <main>@yield('konten')</main>
</body>
</html>

{{-- Halaman anak: resources/views/produk/index.blade.php --}}
@extends('layouts.app')
@section('judul', 'Daftar Produk')
@section('konten')
    <h1>Daftar Produk</h1>

```

```

@forelse($produk as $item)
    <li>{{ $item->nama }} – Rp {{ number_format($item->harga) }}</li>
@empty
    <p>Belum ada produk.</p>
@endforelse
@endsection

```

Direktif Blade yang paling sering dipakai:

Direktif	Fungsi
{{ \$var }}	Tampilkan variabel (auto-escape XSS)
{!! \$html !!}	Tampilkan HTML mentah
@if / @endif	Kondisi percabangan
@foreach / @endforeach	Perulangan dengan \$loop helper
@forelse / @empty	Foreach + handler data kosong
@extends('layout')	Pakai layout induk
@section / @endsection	Definisi konten untuk @yield
@yield('slot')	Tempat konten anak di layout
@auth / @guest	Kondisi berdasarkan status login
@csrf	Token keamanan form (wajib!)
@method('PUT')	Override HTTP method di form
@error('field')	Tampilkan pesan error validasi

Langkah 3.1 — Migration

Migration adalah cara Laravel membuat dan mengelola struktur tabel database lewat kode PHP.

```
# Buat migration
php artisan make:migration create_produk_table

# Jalankan migration
php artisan migrate

# Rollback migration terakhir
php artisan migrate:rollback

# Reset semua & migrate ulang
php artisan migrate:fresh

// Contoh migration: database/migrations/xxx_create_produk_table.php
public function up(): void
{
    Schema::create('produk', function (Blueprint $table) {
        $table->id();
        $table->string('nama');
        $table->text('deskripsi')->nullable();
        $table->decimal('harga', 12, 2);
        $table->integer('stok')->default(0);
        $table->boolean('aktif')->default(true);
        $table->foreignId('kategori_id')->constrained();
        $table->timestamps(); // created_at & updated_at
    });
}
```

Langkah 3.2 — Model Eloquent

Setiap tabel database direpresentasikan oleh sebuah Model. Konvensi: Model Produk → tabel produk.

```
// app/Models/Produk.php
class Produk extends Model
{
    protected $fillable = [
        'nama', 'deskripsi', 'harga', 'stok', 'aktif', 'kategori_id'
    ];
    protected $casts = [
        'harga' => 'decimal:2',
        'aktif' => 'boolean',
    ];
}
```

Operasi CRUD dengan Eloquent:

```
// CREATE
$produk = Produk::create(['nama' => 'Kopi Arabika', 'harga' => 45000]);

// READ
$semua = Produk::all();
```

```

$satu    = Produk::find(1);           // null jika tidak ada
$atau404 = Produk::findOrFail(1);     // 404 jika tidak ada
$filter  = Produk::where('aktif', true)->orderBy('nama')->get();
$cari    = Produk::where('nama', 'LIKE', "%{$keyword}%")->get();
$page    = Produk::paginate(10);     // dengan pagination

// UPDATE
$produk = Produk::find(1);
$produk->update(['nama' => 'Kopi Robusta']);

// DELETE
Produk::find(1)->delete();

// AGREGASI
Produk::count();           // jumlah record
Produk::sum('stok');       // total stok
Produk::avg('harga');     // rata-rata harga
Produk::max('harga');     // harga tertinggi

```

Langkah 3.3 — Relasi Database

Eloquent menyederhanakan relasi antar tabel menjadi method yang mudah dipanggil.

```

// hasMany: User punya banyak Order
class User extends Model {
    public function orders() {
        return $this->hasMany(Order::class);
    }
}

$orders = User::find(1)->orders; // semua order user ini

// belongsTo: Order milik satu User
class Order extends Model {
    public function user() {
        return $this->belongsTo(User::class);
    }
}

$nama = Order::find(1)->user->nama;

// belongsToMany: Produk punya banyak Tag (dan sebaliknya)
class Produk extends Model {
    public function tags() {
        return $this->belongsToMany(Tag::class); // butuh tabel pivot: produk_tag
    }
}

// Eager Loading — hindari N+1 query problem!
$produk = Produk::with(['kategori', 'tags'])->get();

```

■ Selalu gunakan `with()` saat mengakses relasi di dalam loop, untuk menghindari ratusan query tidak perlu.

Langkah 4.1 — Authentication (Laravel Breeze)

Laravel Breeze menyediakan sistem login, register, dan reset password yang siap pakai.

```
# Install Laravel Breeze
composer require laravel/breeze --dev

# Install dengan stack Blade (paling sederhana)
php artisan breeze:install blade

# Install dependencies & jalankan migration
npm install && npm run dev
php artisan migrate
```

Setelah instalasi, route berikut otomatis tersedia:

- /login — Halaman login
- /register — Halaman registrasi
- /forgot-password — Lupa password
- /dashboard — Halaman setelah login (protected)

```
// Lindungi route dengan middleware auth
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index']);
    Route::resource('produk', ProdukController::class);
});

// Di Blade: tampilkan data user yang login
@auth
    <p>Selamat datang, {{ Auth::user()->name }}!</p>
    <a href="/logout">Keluar</a>
@endauth
```

Langkah 4.2 — Middleware

Middleware adalah filter yang berjalan sebelum request masuk ke Controller.

```
# Buat middleware baru
php artisan make:middleware CekAdmin

// app/Http/Middleware/CekAdmin.php
public function handle(Request $request, Closure $next)
{
    if (!Auth::user()->is_admin) {
        return redirect('/dashboard')
            ->with('error', 'Akses ditolak!');
    }
    return $next($request);
}

// Pakai di route
Route::middleware(['auth', 'cek.admin'])->group(function () {
    Route::get('/admin', [AdminController::class, 'index']);
});
```

```
});
```

Langkah 4.3 — File Upload & Storage

```
// Controller: simpan file upload
public function store(Request $request)
{
    $request->validate([
        'foto' => 'required|image|mimes:jpg,png|max:2048',
    ]);

    $path = $request->file('foto')->store('produk', 'public');
    // File tersimpan di storage/app/public/produk/

    Produk::create([
        'nama' => $request->nama,
        'foto' => $path,
    ]);
}

// Blade: tampilkan gambar


// Jalankan sekali untuk symlink storage:
// php artisan storage:link
```

Langkah 5.1 — CRUD Application Lengkap

Gabungkan semua yang sudah dipelajari untuk membuat aplikasi CRUD penuh.

Urutan pembuatan yang direkomendasikan:

1. php artisan make:model Produk -mcr (buat Model + Migration + Controller sekaligus)
2. Isi kolom di file migration, lalu php artisan migrate
3. Definisi \$fillable di Model
4. Tambahkan Route::resource('produk', ProdukController::class) di web.php
5. Isi 7 method di Controller (index, create, store, show, edit, update, destroy)
6. Buat file Blade view untuk setiap method (index, create, edit, show)
7. Test semua fitur di browser

Langkah 5.2 — Search & Pagination

```
// Controller: index() dengan search & pagination
public function index(Request $request)
{
    $query = Produk::query();

    if ($request->filled('cari')) {
        $query->where('nama', 'LIKE', '%' . $request->cari . '%');
    }

    $produk = $query->latest()->paginate(10)->withQueryString();

    return view('produk.index', compact('produk'));
}

{{-- Blade: form pencarian + pagination --}}
<form method="GET">
    <input name="cari" value="{{ request('cari') }}" placeholder="Cari produk...">
    <button type="submit">Cari</button>
</form>

@forelse($produk as $item)
    <tr><td>{{ $item->nama }}</td></tr>
@empty
    <tr><td>Tidak ada produk.</td></tr>
@endforelse

{{-- Link halaman otomatis --}}
{{ $produk->links() }}
```

Langkah 5.3 — Tips Deploy ke Hosting

```
# Sebelum deploy ke production:
php artisan config:cache    # cache konfigurasi
php artisan route:cache     # cache routing
php artisan view:cache      # cache Blade
```

```
# Set di .env production:
APP_ENV=production
APP_DEBUG=false

# Generate app key jika belum ada:
php artisan key:generate
```

Platform hosting yang populer untuk Laravel:

- Shared Hosting (Niagahoster, Dewaweb) — paling murah, setup manual
- Railway / Render — platform modern, free tier tersedia
- VPS (DigitalOcean, Vultr) — kontrol penuh, butuh skill server
- Laravel Forge — tool resmi untuk deploy Laravel ke VPS dengan mudah

Ringkasan Perintah Artisan Paling Penting

Perintah	Fungsi
<code>php artisan serve</code>	Jalankan development server
<code>php artisan migrate</code>	Jalankan semua migration pending
<code>php artisan migrate:fresh</code>	Hapus semua tabel & migrate ulang
<code>php artisan make:model X -mcr</code>	Buat Model + Migration + Controller + Resource
<code>php artisan make:controller X --resource</code>	Buat resource controller dengan 7 method
<code>php artisan make:migration create_X_table</code>	Buat file migration baru
<code>php artisan make:middleware X</code>	Buat middleware baru
<code>php artisan route:list</code>	Lihat semua route yang terdaftar
<code>php artisan tinker</code>	REPL interaktif untuk test Eloquent
<code>php artisan storage:link</code>	Buat symlink untuk public storage
<code>php artisan breeze:install blade</code>	Install autentikasi Breeze dengan Blade
<code>php artisan config:cache</code>	Cache konfigurasi untuk production
<code>php artisan key:generate</code>	Generate APP_KEY baru

■ Selamat belajar Laravel 12! Konsistensi adalah kunci — belajar 1-2 jam setiap hari lebih efektif daripada belajar marathon seminggu sekali. ■

Panduan ini dibuat dengan Claude · [claude.ai](#)